

Dein Specter-Handbuch

So bedienst du deine Sicherheits-Software - Schritt für Schritt, ohne Vorwissen.

Inhalt

- | | |
|--|--|
| 1. Was ist Specter? | 6. Ein Auftrag Schritt für Schritt |
| 2. Die goldene Regel: defensiv & im Rahmen | 7. So führst du ein Kundengespräch |
| 3. Was Specter alles prüft | 8. Was du sagen darfst - und was nicht |
| 4. Erste Einrichtung | 9. Häufige Fragen |
| 5. Welche Daten du beim Kunden brauchst | 10. Spickzettel |

1. Was ist Specter?

Specter ist dein **automatischer, defensiver Sicherheits-Prüfer**. Er schaut sich die IT einer Firma an, findet Schwachstellen und schreibt einen verständlichen Bericht mit konkreten Empfehlungen - so wie es große Sicherheitsfirmen tun, nur günstiger und schneller.

Merksatz: Specter ist wie ein **TÜV für die IT**. Er repariert nichts selbst und bricht nirgends ein - er prüft, dokumentiert und empfiehlt.

Das Besondere: Specter arbeitet **offline**. Er wertet Export-Dateien aus, die der Kunde bereitstellt. Er verbindet sich nicht heimlich mit fremden Systemen. Das macht ihn sicher - und rechtlich sauber.

2. Die goldene Regel: defensiv & im Rahmen

Wichtig fürs Vertrauen & fürs Gesetz (§202a-c StGB, DSGVO):

Specter führt **keine Angriffe** aus. Keine Passwörter knacken, keine Daten stehlen, nichts zerstören, keine Systeme lahmlegen. Nur prüfen, nur im vereinbarten Rahmen (Scope), nur mit schriftlicher Freigabe.

- Aktive Netzwerk-Scanner sind **standardmäßig aus** und müssen pro Auftrag freigeschaltet werden.
- Alles außerhalb des Rahmens wird von der Software **automatisch verweigert** (fail-closed).
- Jede Aktion wird protokolliert - du kannst jederzeit belegen, was gemacht wurde.

Genau das ist dein Verkaufsargument: **Firmen lassen dich an ihre Systeme, weil Specter nachweislich nichts kaputt macht und nichts abfließt.**

3. Was Specter alles prüft

Vierzehn Prüf-Module („Analyzer“) decken die Bereiche ab, die im Mittelstand wirklich zu Schäden führen:

Modul	Was es findet	Warum es zählt
<code>analyze_ad</code>	Active Directory: schwache Passwort-Regeln, zu viele Admins, veraltete Konten, Kerberoasting/Golden-Ticket-Risiken.	Das Windows-Herz fast jeder Firma - hier hängt alles dran.
<code>analyze_exchange</code>	Exchange/Outlook: veraltete Server-Version, extern erreichbares ECP, schwache TLS-Einstellungen.	Mail-Server sind ein beliebtes Einfallstor (ProxyShell & Co.).
<code>analyze_entra</code>	Microsoft 365 / Entra ID: fehlende MFA, Legacy-Anmeldung, zu viele globale Admins, offene Freigaben.	Cloud-Identitäten sind das neue Passwort zur ganzen Firma.
<code>analyze_aws</code>	AWS: Root ohne MFA, offene S3-Buckets, zu weite Rechte, offene Security-Groups.	Ein offener Bucket = Datenleck in Sekunden.
<code>analyze_azure</code>	Azure: öffentliche Speicher, offene Ports, VMs mit alter Software, Key Vaults ohne Schutz.	Cloud-Fehlkonfiguration ist die häufigste Cloud-Lücke.
<code>analyze_email_security</code>	E-Mail-Schutz (SPF/DKIM/DMARC): kann jemand in eurem Namen Mails fälschen?	Schützt vor CEO-Fraud - der teuerste Betrug im Mittelstand.
<code>analyze_dns</code>	DNS-Sicherheit: fehlendes DNSSEC, fehlende CAA-Records, offener Zonentransfer (AXFR), Wildcard, dangling CNAME.	DNS ist das Adressbuch der Firma - manipuliert es jemand, landet Verkehr beim Angreifer.
<code>analyze_dependencies</code>	Software-Bibliotheken: bekannte Lücken (Log4Shell-Klasse), veraltete oder ungepinnte Pakete.	Fremd-Code steckt überall - und altert schnell.
<code>analyze_firewall</code>	Firewall/VPN: Any-Any-Regeln, offenes RDP/SSH aus dem Internet, VPN ohne MFA.	Offenes RDP ist die Ransomware-Tür Nummer eins.
<code>analyze_tls</code>	TLS/Zertifikate: abgelaufen, schwache Verschlüsselung, alte Protokolle.	Für jeden von außen sichtbar - ein schneller Vertrauensverlust.

<code>analyze_http_headers</code>	Web-Sicherheit: fehlende Schutz-Header (HSTS/CSP/X-Frame-Options), unsichere Cookies, verräterische Server-Banner.	Websites sind das Schaufenster - hier fällt Nachlässigkeit sofort auf.
<code>analyze_backup</code>	Backup/Ransomware-Resilienz: 3-2-1-Regel, offline/unveränderbare Kopien, getestete Wiederherstellung.	Entscheidet, ob ihr eine Ransomware überlebt - Versicherer-Prüfpunkt Nr. 1.
<code>analyze_database</code>	Datenbanken: öffentlich erreichbare Ports, fehlende Authentifizierung (Redis/Mongo), Default-Zugangsdaten, Transport ohne TLS.	In Datenbanken liegen die Kronjuwelen - offen erreichbar sind sie ein Selbstbedienungsladen.
<code>analyze_container</code>	Container/Docker: privilegierte Container, gemountetes docker.sock, Host-Networking, gefährliche Capabilities, root, :latest.	Ein schlecht konfigurierter Container ist der direkte Weg vom Dienst zum Host.

Zusätzlich: automatische **Angriffspfad-Analyse** (welche kleinen Lücken zusammen gefährlich werden), ein **CVSS-Score** je Fund, **BSI-IT-Grundschutz**-Zuordnung und ein **Nachtest** (was wurde behoben?).

4. Erste Einrichtung

Einmalig auf deinem Rechner (oder Server):

```
git clone <repo>
cd Specter-
pip install -r requirements.txt
```

Zum Ausprobieren ohne echte Kundendaten gibt es eine fertige Demo:

```
python examples/run_demo.py
```

Die Demo startet einen kleinen Test-Server auf deinem eigenen Rechner (127.0.0.1) und zeigt den kompletten Ablauf - völlig gefahrlos.

5. Welche Daten du beim Kunden brauchst

Du brauchst **keinen** Vollzugriff. Du bittest den Kunden um **Export-Dateien** (JSON) - lesend, harmlos, schnell erstellt:

Bereich	Beispiel-Export
Active Directory	Benutzer-/Richtlinien-Export bzw. BloodHound-Daten
Microsoft 365 / Entra	MFA-/Conditional-Access-Report
AWS / Azure	IAM-/Storage-/Netzwerk-Export
E-Mail / DNS	DNS-Einträge (SPF/DKIM/DMARC, DNSSEC/CAA)
Software	requirements.txt / package.json / SBOM
Firewall / VPN	Regelwerk-Export
TLS / Web	Zertifikats-/Protokoll-Übersicht, HTTP-Header
Datenbanken	Port-/Auth-/TLS-Übersicht der DB-Dienste
Container	<code>docker inspect</code> -Ausgabe
Backup	Kurzer Fragebogen zur Backup-Strategie

Tipp: In `examples/data/` liegt für jeden Bereich eine Beispiel-Datei. Zeig sie dem Kunden - dann weiß die IT-Abteilung sofort, was du brauchst.

6. Ein Auftrag Schritt für Schritt

1. **Rahmen festlegen:** Trage die erlaubten Ziele und Pfade in eine `scope.yaml` ein (Vorlage: `scope.example.yaml`). Nur was hier steht, wird geprüft.
2. **Daten sammeln:** Lege die Export-Dateien des Kunden in einen Ordner im Rahmen.
3. **Prüfen:** Starte Specter mit dem Ziel des Auftrags. Ohne KI-Schlüssel laufen die Analyzer direkt; mit Schlüssel steuert das KI-Modell den Ablauf.
4. **Bericht erzeugen:** Specter schreibt einen Markdown- und einen schönen HTML-Bericht.
5. **PDF & Übergabe:** HTML im Browser öffnen → „Drucken → Als PDF speichern“ → fertiges Kunden-PDF.

6. **Nachtest:** Nach der Behebung erneut prüfen - der Bericht zeigt, was jetzt behoben ist.

```
python main.py --scope scope.yaml \  
  --objective "Prüfe die bereitgestellten Exporte in ./kundendaten"
```

7. So führst du ein Kundengespräch

Ein einfacher Ablauf, den du auswendig können kannst:

1. **Sorge ansprechen:** „Die meisten Schäden im Mittelstand kommen über E-Mail-Betrug, offenes RDP und fehlende Backups. Genau das prüfe ich.“
2. **Sicherheit betonen:** „Ich greife nichts an und nehme nichts mit. Ich werte nur Export-Dateien aus, die Sie mir geben.“
3. **Nutzen zeigen:** „Sie bekommen einen verständlichen Bericht mit Prioritäten - und einen Nachweis für Ihre Cyber-Versicherung.“
4. **Kleiner Einstieg:** Biete einen günstigen Erst-Check (z. B. E-Mail-Schutz + Backup) als Türöffner an.

Versicherungs-Argument: Viele Cyber-Policen verlangen MFA, getestete Backups und Patch-Management. Specter prüft genau diese Punkte - dein Bericht hilft dem Kunden, versicherbar zu bleiben.

8. Was du sagen darfst - und was nicht

So sagst du es (ehrlich)	So bitte nicht
„Ich prüfe defensiv und dokumentiere.“	„Ich hacke Ihre Firma.“
„Ich werte Ihre Export-Daten aus.“	„Ich brauche Admin-Zugang zu allem.“
„Der Bericht ist eine fachkundige Orientierung.“	„Das ist ein zertifiziertes Testat.“
„Ich halte mich strikt an den vereinbarten Rahmen.“	„Ich schau mal überall rein.“

Immer schriftliche Freigabe einholen, bevor du irgendetwas prüfst. Ohne Auftrag kein Zugriff - das schützt dich und den Kunden.

9. Häufige Fragen

Brauche ich einen KI-Schlüssel?

Nein. Die Analyzer laufen auch ohne. Mit einem Schlüssel (z. B. Fable 5) steuert das KI-Modell den Ablauf und korreliert Funde zusätzlich.

Kann Specter etwas kaputt machen?

Nein. Es führt keine Angriffe aus und ändert keine Kundensysteme. Aktive Scanner sind aus, bis du sie bewusst freischaltest.

Was, wenn ich mich nicht sicher bin?

Im Zweifel weniger prüfen, nicht mehr. Halte dich an die `scope.yaml` und frage beim Kunden nach.

Wie überzeuge ich technische Ansprechpartner?

Zeig die BSI-IT-Grundschutz-Zuordnung und den CVSS-Score im Bericht - das ist die Sprache, die IT-Abteilungen kennen.

10. Spickzettel

Aufgabe	Befehl
Demo ansehen	<code>python examples/run_demo.py</code>
Selbst-Check von Specter	<code>python examples/self_audit.py</code>
Echter Auftrag	<code>python main.py --scope scope.yaml --objective "..."</code>
Tests laufen lassen	<code>python -m pytest</code>
Dieses Handbuch bauen	<code>python examples/build_handbook.py</code>

Du schaffst das. Fang mit der Demo an, dann mit einem kleinen Erst-Check bei einem Kunden, den du kennst. Jeder Auftrag macht dich sicherer.